

SIMATS ENGINEERING

ASSESSMENT TOOL 1

COURSE NAME: OBJECT ORIENTED ANALYSIS AND DESIGN
COURSE CODE: CSA1105

COURSE OUTCOME COVERED

CO5: Analyze object-oriented software using quality assurance and usability testing Techniques (BL4,BL5)
Assessment Tool Weightage: 35%

QUESTIONS: 5

Total Marks:35

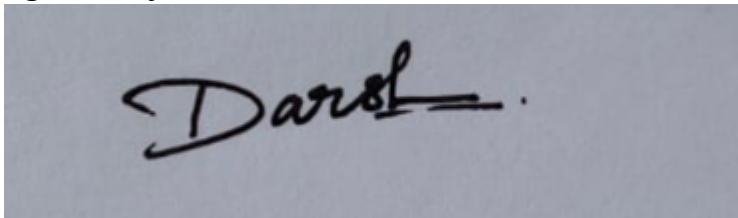
Annexure A

Constraint-Based Problem Solving

Code of Conduct:

I Darshinie Karani.V.M (192520033) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

A rectangular box containing a handwritten signature in black ink. The signature appears to be 'Darsh' followed by a horizontal line and a dot.

RUBRICS FOR CALCULATION:

Constraint-Based Problem Solving Assessment Rubric

Criteria	Weightage	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Problem Understanding	25%	Clearly defines the problem and identifies all relevant constraints accurately	Identifies most constraints with minor gaps	Partial understanding; misses key constraints	Fails to identify constraints correctly
Constraint Analysis	25%	Analyzes constraints effectively and prioritizes them logically	Provides reasonable analysis with some prioritization	Limited analysis; weak prioritization	No meaningful analysis or prioritization
Solution Development	25%	Develops optimal and feasible solutions satisfying all constraints	Develops workable solutions with minor limitations	Solutions partially satisfy constraints	Solutions do not meet constraints
Application of Concepts	15%	Effectively applies appropriate concepts, methods, or tools	Applies concepts with minor errors	Limited application; requires guidance	Unable to apply relevant concepts
Justification	10%	Provides strong justification for decisions with logical reasoning	Provides justification with some clarity	Weak or unclear justification	No justification provided

Annexure A

Constraint-Based Problem Solving

Q1. Scenario: Digital Wallet System

System Components:

User Registration, Wallet Management, Money Transfer, Transaction History

Given Constraints:

- Transaction response time < **3 seconds**
- Biometric or OTP authentication **required**
- Support **10,000+ concurrent users**
- Transaction records must be accurate and consistent

Question:

Analyze the digital wallet system using appropriate **quality assurance and software testing techniques** under the given constraints. Identify potential defects related to security, concurrency, performance, and data integrity. Propose suitable testing strategies and justify how they ensure system reliability and quality.

(BL4)

Q2. Scenario: Online Grocery Shopping Application

System Features:

Product Browsing, Product Search, Shopping Cart, Checkout, Order Tracking

Usability Constraints:

- Product search results displayed within **2 seconds**
- Checkout completed within **4 steps**
- Interface must support **mobile and desktop devices**
- Product categories should be easily accessible

Question:

Apply appropriate **usability testing techniques** to evaluate the grocery shopping application. Identify potential usability problems and recommend interface improvements that satisfy the given constraints. Explain how the proposed improvements enhance customer satisfaction, accessibility, and task efficiency.

(BL4)

Q3. Scenario: University Examination Management System

Modules:

Student Registration, Question Bank, Online Examination, Evaluation, Result Processing

QA Constraints:

- Examination results must be **accurate and consistent**
- System must support **5,000+ simultaneous students**

- Examination service must remain available throughout the examination period
- Unauthorized access to examination content must be prevented

Question:

Use **quality assurance and testing practices** to analyze the system under the given constraints. Identify critical test scenarios related to accuracy, performance, security, availability, and concurrency. Recommend suitable testing techniques and explain how they help prevent defects and improve system reliability.

(BL5)

Q4. Scenario: Smart Healthcare Appointment System

System Features:

Patient Registration, Doctor Search, Appointment Booking, Appointment Cancellation, Notifications

Usability Constraints:

- Appointment booking should require **a maximum of 3 major steps**
- Interface must be accessible to users with different levels of technical experience
- Appointment availability should be clearly displayed
- Confirmation messages must be easy to understand

Question:

Conduct a **usability evaluation** of the healthcare appointment system using suitable testing methods. Analyze potential usability and accessibility problems and recommend design improvements that satisfy the specified constraints. Explain how the improvements reduce user errors and improve overall user satisfaction.

(BL5)

Q5. Scenario: Online Railway Ticket Reservation System

Modules:

User Authentication, Train Search, Seat Availability, Passenger Details, Ticket Booking, Payment

QA & Usability Constraints:

- Duplicate seat booking must be prevented
- System must handle **peak festival-season traffic**
- Payment failures must not result in incorrect ticket confirmation
- Booking interface should minimize user errors

Question:

Apply **quality assurance and usability testing techniques** to evaluate the railway reservation system under the specified constraints. Identify major risks related to concurrency, transaction processing, performance, security, and usability. Propose suitable testing strategies and explain how they improve reliability, performance, and user experience.

(BL4)

ANSWERS

1.

The Digital Wallet System consists of four major components: User Registration, Wallet Management, Money Transfer, and Transaction History. The system must satisfy four important constraints: transaction response time should be less than 3 seconds, biometric or OTP authentication is required, the system should support more than 10,000 concurrent users, and transaction records must remain accurate and consistent.

Problem Understanding and Constraints

The major constraints of the digital wallet are:

- Performance: Every transaction should be completed with a response time of less than 3 seconds.
- Security: Users must be authenticated using OTP or biometric verification.
- Concurrency: The system must support 10,000+ users simultaneously.
- Data Integrity: Transaction records, wallet balances, and transaction history must always remain accurate.
- Reliability: Failed transactions should not result in incorrect balance updates.
- Usability: Users should be able to register, transfer money and view transactions easily.

Security Testing

Security testing is performed to identify unauthorized access and authentication-related defects.

Test cases:

- Enter a valid OTP and verify successful authentication.
- Enter an incorrect OTP and verify that access is denied.
- Test expired OTPs.
- Test multiple incorrect OTP attempts.
- Verify biometric authentication.
- Attempt to access another user's wallet without authorization.
- Check whether sensitive transaction information is protected.

Possible defects:

- Unauthorized access to wallet.
- OTP bypass.
- Weak authentication.
- Session-related vulnerabilities.
- Exposure of sensitive transaction details.

Solution: Strong authentication, session management and access control should be implemented.

Performance Testing

Performance testing checks whether the system satisfies the less-than-3-second response-time constraint.

Testing techniques:

- Load Testing: Simulate 10,000+ users performing transactions.
- Stress Testing: Increase the number of users beyond the expected capacity.
- Response-Time Testing: Measure the time required to complete each transaction.
- Spike Testing: Suddenly increase the number of users and observe system behaviour.

Possible defects:

- Slow money transfers.
- Server overload.
- Database bottlenecks.
- Transaction timeout.
- System crash under heavy load.

Expected result: Transactions should respond within the specified 3-second limit under expected operating conditions.

Concurrency Testing

Since the system should support 10,000+ concurrent users, concurrency testing is important.

Important scenarios:

- Two users transfer money simultaneously.
- Multiple users access their wallets at the same time.
- Two transactions attempt to modify the same account simultaneously.
- Multiple users perform transactions on the same wallet.

Possible defects:

- Race conditions.
- Incorrect wallet balance.
- Duplicate transactions.
- Double spending.
- Lost transaction records.

Solution: Database transactions, synchronization and appropriate locking mechanisms should be used.

Data Integrity Testing

Data integrity testing ensures that transaction information remains accurate and consistent.

For example, if User A transfers ₹1,000 to User B:

User A balance → decreases by ₹1,000

User B balance → increases by ₹1,000

The transaction history should also contain the correct transaction details.

Test cases:

- Successful transfer.
- Transfer with insufficient balance.
- Transfer failure during processing.
- Network failure during transaction.
- Database failure during transaction.
- Duplicate transaction request.

Expected behaviour:

If a transaction fails, the system should perform a rollback so that an incorrect amount is not deducted or credited.

Functional Testing

Functional testing verifies whether each feature works according to its requirements.

Function	Test Scenario
User Registration	Register with valid and invalid details
Login	Test valid and invalid authentication
Wallet Management	Check wallet balance and details
Money Transfer	Transfer valid amount
Insufficient Balance	Attempt transfer exceeding balance
OTP	Test valid, invalid and expired OTP
Transaction History	Verify correct transaction records

This helps identify defects in the individual modules of the wallet system.

Regression Testing

Regression testing should be performed whenever modifications are made to the system.

For example, if the OTP authentication module is modified, previously working functions such as:

- Login
- Wallet access
- Money transfer
- Transaction history

should be tested again.

This ensures that a new change has not introduced defects into existing functionality.

Defect Identification and Prevention

Area	Possible Defect	Testing Technique
Security	Unauthorized wallet access	Security testing
Authentication	Invalid OTP accepted	Security/functional testing
Performance	Response time > 3 seconds	Load testing
Concurrency	Duplicate transaction	Concurrency testing
Data Integrity	Incorrect balance	Data integrity testing
Reliability	Failed transaction still deducted money	Recovery testing
Functionality	Incorrect transaction history	Functional testing

Overall Quality Assurance Strategy

A complete QA strategy should follow these stages:

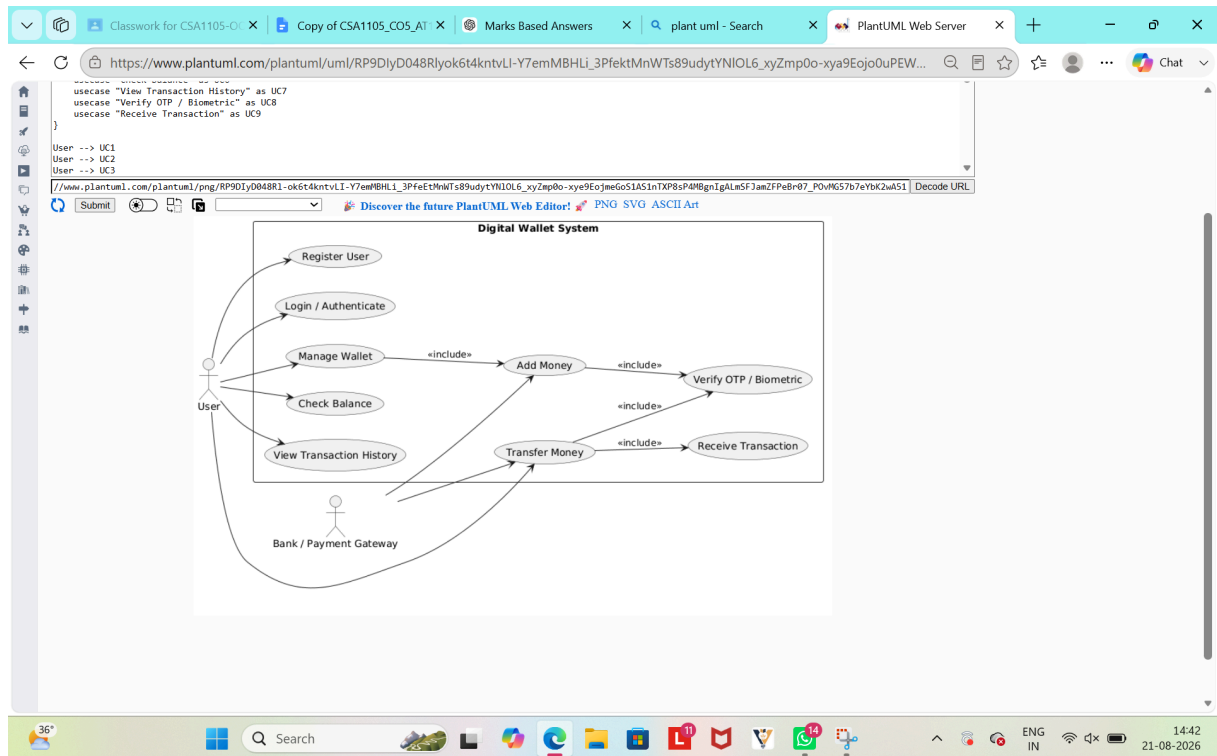
Requirement Analysis → Test Planning → Test Case Design → Functional Testing → Security Testing → Performance Testing → Concurrency Testing → Data Integrity Testing → Regression Testing → Defect Fixing → Retesting

This systematic approach helps ensure that all specified constraints are tested before deployment.

Justification

The proposed testing techniques directly address the given constraints:

- Security testing ensures biometric/OTP authentication and prevents unauthorized access.
- Performance testing verifies the 3-second response requirement.
- Concurrency testing ensures reliable operation for 10,000+ simultaneous users.
- Data integrity testing prevents incorrect balances and transaction records.
- Functional testing verifies that all wallet features operate correctly.
- Regression testing ensures that modifications do not introduce new defects.



Conclusion

Therefore, a combination of functional, security, performance, concurrency, data integrity and regression testing is required for the Digital Wallet System. These techniques help identify critical defects, satisfy the specified constraints and ensure that the system is secure, accurate, reliable, scalable and efficient. The assessment specifically asks for defects related to security, concurrency, performance and data integrity and for testing strategies that ensure reliability and quality.

2.

The Smart Healthcare Appointment System consists of Patient Registration, Doctor Search, Appointment Booking, Appointment Cancellation, and Notifications. The major usability constraints are that appointment booking must require a maximum of 3 major steps, the interface must support users with different levels of technical experience, appointment availability must be clearly displayed, and confirmation messages must be easy to understand.

Usability Evaluation

Usability testing should be conducted by observing users while they perform common tasks such as:

- Registering as a patient.
- Searching for a doctor.
- Checking available appointment slots.
- Booking an appointment.
- Cancelling an appointment.
- Understanding the confirmation notification.

The evaluator should measure task completion, time taken, number of errors, navigation difficulty, and user feedback.

Suitable Usability Testing Methods

a) User Testing

Real users with different levels of technical experience should perform appointment-related tasks.

The evaluator observes:

- Whether users can easily find a doctor.
- Whether they can identify available slots.
- Whether they can complete booking within 3 steps.
- Whether they understand confirmation and error messages.

b) Task-Based Testing

Give users specific tasks such as:

"Find a doctor, select an available time slot and book an appointment."

The system should allow this task to be completed in three major steps or fewer.

c) Accessibility Testing

Test whether users with different technical abilities can understand and operate the system.

Check:

- Text readability.
- Button visibility.
- Clear labels.
- Simple navigation.
- Easy-to-understand messages.

d) User Feedback

After testing, collect feedback regarding:

- Ease of use.
- Navigation.
- Clarity of appointment information.
- Booking process.
- Overall satisfaction.

Potential Usability Problems

The following problems may occur:

1. Too many booking steps
Users may become confused if they have to move through several screens.
2. Unclear appointment availability
Users may not know which doctors or time slots are available.
3. Complex interface
Users with limited technical experience may have difficulty navigating the system.
4. Confusing confirmation messages
Technical or lengthy messages may cause users to misunderstand whether the appointment was successfully booked.
5. Poor button placement
Users may accidentally select the wrong option, such as cancellation instead of booking.
6. Difficult cancellation process
Users may not understand how to cancel an appointment.

Accessibility Problems

Potential accessibility issues include:

- Small or difficult-to-read text.
- Poor contrast between text and background.
- Unclear icons without labels.
- Complicated terminology.
- Important information not being clearly visible.
- Buttons that are difficult to identify or operate.

These problems can make the system difficult for users with different levels of technical experience.

Recommended Design Improvements

a) Three-Step Booking Process

The booking process should be simplified to:

Step 1: Select Doctor

↓

Step 2: Select Available Date & Time

↓

Step 3: Confirm Appointment

This directly satisfies the maximum 3 major steps constraint.

b) Clear Availability Display

Available appointment slots should be displayed clearly using:

- Doctor name
- Date
- Time
- Availability status

Unavailable slots should not be selectable.

c) Simple Interface

Use:

- Simple menus.
- Clearly labelled buttons.
- Familiar icons with text labels.
- Consistent page layout.
- Simple language.

d) Clear Confirmation Messages

After successful booking, display a simple message such as:

"Appointment booked successfully."

The system should also show important details such as doctor, date and time.

For cancellation:

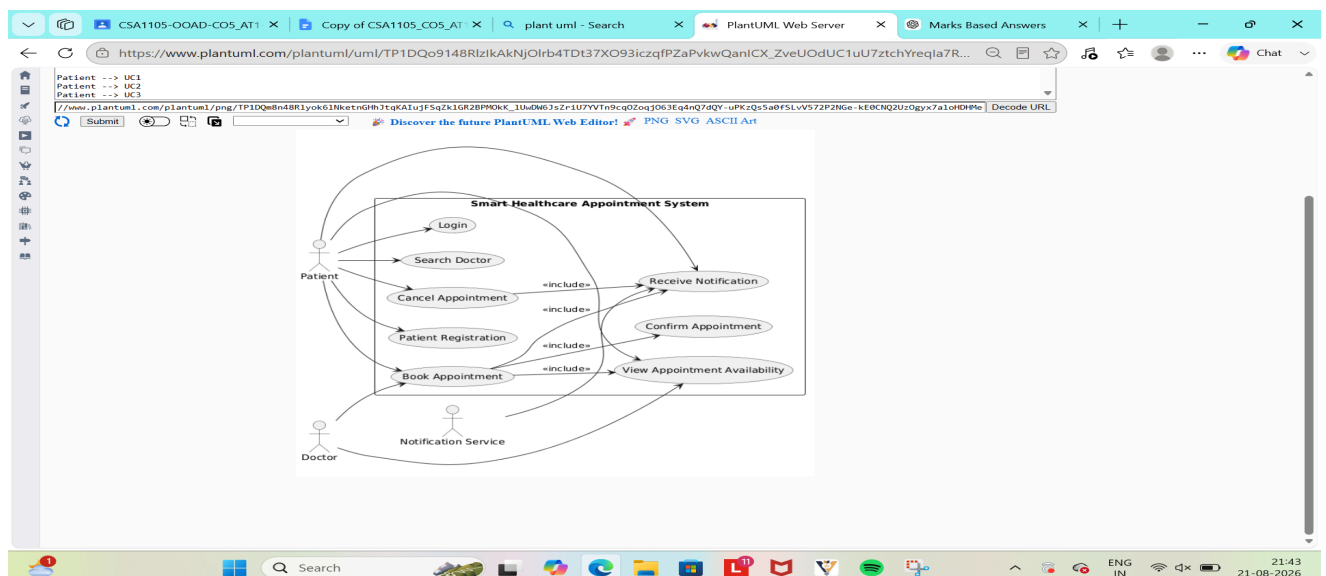
"Your appointment has been cancelled successfully."

e) Error Prevention

The system should:

- Prevent selection of unavailable slots.
- Ask for confirmation before cancellation.
- Validate patient details before booking.
- Clearly indicate required fields.
- Provide understandable error messages.

Diagram:



How the Improvements Reduce User Errors

The improvements reduce errors by making the user's actions simple, visible and predictable.

- Three-step booking reduces navigation errors.
- Clear availability prevents users from selecting unavailable slots.
- Simple labels reduce misunderstanding.
- Confirmation messages help users know whether their action was successful.
- Confirmation before cancellation prevents accidental cancellations.
- Input validation prevents incorrect patient information.

Effect on User Satisfaction

These improvements improve satisfaction because users can:

- Book appointments quickly.
- Easily identify available doctors and time slots.
- Complete tasks with fewer clicks.
- Understand system messages immediately.
- Avoid unnecessary errors and confusion.
- Use the system even with limited technical experience.

Conclusion

A combination of user testing, task-based testing, accessibility testing and user feedback can effectively evaluate the healthcare appointment system. A three-step booking process, clear availability display, simple interface, accessible design and understandable confirmation messages satisfy the given usability constraints. These improvements reduce user errors, improve task efficiency and increase overall user satisfaction.